Assignment 1 - PS1: Smart Waste Collection Agent

Design Document

1. Problem Statement

Bengaluru's Smart City waste management program needs a robot to travel between collection centers and processing units as cheaply as possible. The road network is represented as a weighted undirected graph -- nodes are locations and each edge carries a cost reflecting fuel or travel time. Given a source and a destination, the task is to find the least-cost path between them.

This is a well-suited problem for informed search: the state space is small and discrete, costs are non-negative, and we can construct a meaningful heuristic from the graph structure. We chose IDA* as the algorithm because it is both optimal and memory-efficient -- important qualities when the robot has limited on-board resources.

2. PEAS Analysis

Component	Description
Performance	Find the minimum-cost path from source to destination. Lower total travel cost means better performance.
Environment	A static, weighted undirected graph of Bengaluru waste management locations. Edge weights are positive real numbers (fuel cost or distance).
Actuators	The robot moves along an edge from its current node to an adjacent node.
Sensors	The robot knows its current location, the edges leaving it (with costs), and the identity of the goal node.

The agent is goal-based and operates in a fully observable, deterministic, static, discrete environment. There is no uncertainty -- the graph is known upfront and does not change during the search.

3. Heuristic Function

The central design question for IDA* is: how do we estimate the remaining cost from a node to the goal without actually solving the problem? The estimate needs to be a lower bound -- never an overestimate -- so that IDA* can safely prune paths while still guaranteeing the optimal solution.

Our answer is to combine two pieces of information we can compute cheaply. First, how many edges at minimum does it take to reach the goal from the current node? That is the BFS hop distance on the unweighted graph. Second, what is the cheapest any single edge in the graph can be? That is min_edge_weight. The product gives a valid lower bound:

```
h(n) = min_hops(n, goal)  x  min_edge_weight
```

This is admissible because every path to the goal must use at least min_hops edges, and each edge costs at least min_edge_weight, so the product can never exceed the true remaining cost. It is also consistent: moving one hop closer can reduce h by at most min_edge_weight, which is always <= the actual edge cost. Both properties ensure IDA* finds the optimal path.

4. Cost Function and Search Mechanics

IDA* evaluates each node using the standard f = g + h formulation. Here, g(n) is the actual cost accumulated along the path from the source to node n, and h(n) is the heuristic estimate of what remains. Together they give an optimistic total cost for any solution passing through n.

```
f(n) = g(n) + h(n)
```

The algorithm starts with a threshold equal to h(source). It runs a depth-first search and prunes any branch where f(n)

exceeds the threshold -- those branches cannot lead to a solution cheaper than what we have already found or estimated. If the search finishes without reaching the goal, the threshold is updated to the smallest f-value that was pruned in that round, and the whole process repeats. Each iteration explores deeper into the cost space until the optimal solution is found.

5. Algorithm Choice -- Why IDA*?

The most obvious alternative to IDA* is A*, which also uses the same f = g + h function and is optimal with an admissible heuristic. The difference is memory. A* keeps the entire frontier (the open list) in a priority queue. In the worst case this grows exponentially with the search depth. IDA* sidesteps this by never storing the frontier at all -- it only remembers the nodes on the current path, which is linear in depth (O(d)).

Dijkstra's algorithm (uniform cost search) is another strong candidate and is discussed in more detail in Section 7. Blind searches like BFS or DFS are not suitable here because they ignore edge costs entirely.

We chose IDA* because the robot is resource-constrained. Paying a small computational cost to re-expand some nodes across iterations is a reasonable trade-off for drastically lower memory usage. And with a good heuristic, the re-expansion overhead is modest in practice.

6. Data Structures Used

PathStack -- a bounded stack that maintains the current DFS path. Its capacity is set to the number of nodes in the graph (the longest possible acyclic path). push() prints a 'stack overflow' message if the stack is full, and pop() prints 'stack underflow' if empty. A companion hash set provides O(1) membership checks for cycle detection (checking if a neighbor is already on the current path).

Adjacency List -- the graph is stored as a Python dictionary mapping each node to a list of (neighbor, weight) tuples. Since roads are bidirectional, every edge is added in both directions. An empty-graph warning is printed if no edges are loaded.

BFS Queue (deque) -- used once during heuristic pre-computation to find hop distances from the goal to all reachable nodes.

7. Implementation Overview

The full solution lives in a single Jupyter notebook: ida_star_waste_collection.ipynb. Input is read from a plain text file that accepts two formats -- space-separated 'NodeA NodeB Weight' and a compact 'AB3' style. The graph is stored as an undirected adjacency list. Before the search starts, the heuristic values for every node are pre-computed in one BFS pass from the goal, so the search itself never needs to re-run BFS.

IDA* is implemented as a recursive DFS that uses the PathStack for the current path. Push/pop operations on the stack correspond to moving deeper or backtracking. When f(n) > threshold the branch is pruned and the exceeded value is returned. The outer loop collects the minimum exceeded value across all pruned branches and uses it as the next threshold. Between iterations the stack is reset to just the source.

The visited sequence is tracked per iteration, giving a clear picture of which nodes were explored in each pass. Output is both printed and written to outputPSXX.txt.

8. Error Handling

The code handles: missing/unreadable input files, empty files, malformed edge lines, non-positive weights, blank source/destination, source or destination not in graph, disconnected graphs (no path), source == destination (trivial case), stack overflow (path exceeds graph size), and stack underflow (pop on empty).

9. Alternate Approach -- Dijkstra's Algorithm

Dijkstra's algorithm is the most natural comparison point. It processes nodes in order of accumulated cost g(n) using a min-heap, and it is guaranteed to find the optimal path in graphs with non-negative weights -- which is exactly our setting.

Aspect	**IDA***	**Dijkstra**
Optimality	Yes (with admissible h)	Yes
Memory	O(d) -- current path only	O(V) -- visited set + priority queue
Guidance	Uses h(n) to prune branches early	Uninformed, expands by cost alone
Node re-expansion	Possible across iterations	None (each node settled once)
Practical fit	Good for memory-limited agents	Good when memory is plentiful

For this problem size either algorithm works fine. We prefer IDA* because it demonstrates informed search with a heuristic, uses negligible memory, and directly addresses the battery-limited robot scenario.

10. Test Cases and Sample Output

We tested with two cases. Case 1 uses real Bengaluru location names (8 edges) and Case 2 uses abstract nodes in compact format (6 edges) to verify the parser handles both input styles.

```
Case 1 -- MG_Road to Yelahanka
  Iteration 1 visited: MG_Road -> Electronic_City -> Whitefield
  Iteration 2 visited: MG_Road -> Electronic_City -> Whitefield -> Yelahanka
  Optimal Path: MG_Road -> Electronic_City -> Whitefield -> Yelahanka
  Cost: 8

Case 2 -- A to E
  Iteration 1 visited: A
  Iteration 2 visited: A -> C
  Iteration 3 visited: A -> C -> D
  Iteration 4 visited: A -> B -> C -> D -> E
  Optimal Path: A -> C -> D -> E
  Cost: 5
```

Both results were verified by hand against the input graph. The notebook also demonstrates seven error-handling scenarios: invalid source/destination, source equals destination, disconnected graph, empty graph, missing input file, and a minimal two-node graph.

11. How to Run

Place ida_star_waste_collection.ipynb and inputPSXX.txt in the same directory. Open the notebook in Jupyter (or VS Code) and run all cells top to bottom. The results print inline and are also saved to outputPSXX.txt in the same folder. To test a different graph, edit inputPSXX.txt and re-run.